

Aufgabenblatt: Listen

Pflichtaufgabe: Listenoperationen mit Slicing

Beschreibung: Python bietet eine elegante Technik um Teile einer Liste (oder eines Strings) in einer einzigen Zeile auszuschneiden, umzukehren oder zu überspringen. Diese Technik nennt sich **Slicing** und ist in der Praxis allgegenwärtig.

Aufgabenstellung: Recherchiere selbständig wie Slicing in Python funktioniert und löse damit folgende Aufgaben. Alle Ausgaben sollen **ohne Schleifen** umgesetzt werden ausschließlich mit Slicing.

Gegeben ist folgende Liste:

```
highscore = [12, 45, 7, 89, 34, 56, 23, 78, 91, 5]
```

1. Gib die ersten drei Einträge aus.
2. Gib die letzten drei Einträge aus.
3. Gib alle Einträge ab dem vierten aus.
4. Gib jeden zweiten Eintrag aus (Index 0, 2, 4, ...).
5. Gib die Liste in umgekehrter Reihenfolge aus.
6. Gib die mittleren fünf Einträge aus (Index 2 bis 6).

Hinweis: Suche nach „Python List Slicing“, die offizielle Python-Dokumentation und viele Tutorials erklären das Konzept gut und mit Beispielen.

Beispielausgabe:

```
Original:      [12, 45, 7, 89, 34, 56, 23, 78, 91, 5]
Erste drei:    [12, 45, 7]
Letzte drei:   [78, 91, 5]
Ab dem vierten: [89, 34, 56, 23, 78, 91, 5]
Jeden zweiten: [12, 7, 34, 23, 91]
Umgekehrt:     [5, 91, 78, 23, 56, 34, 89, 7, 45, 12]
Mittlere fünf: [7, 89, 34, 56, 23]
```

Aufgabe 1: Notenverwaltung

Aufgabenstellung: Schreibe ein Programm, das den Benutzer so lange nach Noten fragt (ganze Zahlen von 1–6), bis er eine leere Eingabe macht. Danach soll das Programm ausgeben:

- Alle eingegebenen Noten als Liste
- Die Anzahl der Noten
- Den Notendurchschnitt
- Die beste und schlechteste Note

Beispielausgabe:

```
Note eingeben (Enter zum Beenden): 2
Note eingeben (Enter zum Beenden): 3
Note eingeben (Enter zum Beenden): 1
Note eingeben (Enter zum Beenden): 4
Note eingeben (Enter zum Beenden):
```

```
Noten:      [2, 3, 1, 4]
Anzahl:     4
Durchschnitt: 2.50
Beste Note: 1
Schlechteste: 4
```

Aufgabe 2: Listen zusammenführen

Beschreibung: Zwei Personen haben je eine Einkaufsliste geschrieben. Jetzt soll daraus eine gemeinsame Liste entstehen ohne doppelte Artikel und alphabetisch sortiert.

Aufgabenstellung: Gegeben sind zwei Einkaufslisten direkt im Code:

```
liste1 = ["Milch", "Brot", "Butter", "Eier"]
liste2 = ["Käse", "Brot", "Mehl", "Milch", "Zucker"]
```

Schreibe ein Programm, das:

- Beide Listen zu einer gemeinsamen Liste zusammenführt
- Doppelte Artikel erkennt und nur einmal behält
- Die Gesamtliste alphabetisch sortiert ausgibt
- Ausgibt, welche Artikel in beiden Listen vorkamen

Beispielausgabe:

```
Liste 1: ['Milch', 'Brot', 'Butter', 'Eier']
Liste 2: ['Käse', 'Brot', 'Mehl', 'Milch', 'Zucker']
```

```
Gemeinsame Einkaufsliste (sortiert):
- Brot
- Butter
- Eier
- Käse
- Mehl
- Milch
```

- Zucker

In beiden Listen vorhanden: ['Brot', 'Milch']

Aufgabe 3: Einkaufsliste verwalten

Beschreibung: Listen lassen sich interaktiv verwalten Elemente hinzufügen, anzeigen und entfernen ist ein klassischer Anwendungsfall.

Aufgabenstellung: Schreibe ein Einkaufslisten-Programm mit einem Menü. Der Benutzer kann zwischen folgenden Optionen wählen:

- 1 – Artikel hinzufügen
- 2 – Artikel entfernen (nach Name)
- 3 – Liste anzeigen
- 4 – Programm beenden

Das Programm soll so lange laufen, bis der Benutzer 4 eingibt. Wenn ein Artikel entfernt werden soll, der nicht in der Liste ist, soll eine passende Meldung ausgegeben werden.

Beispielausgabe:

```
=== Einkaufsliste ===
1 - Artikel hinzufügen
2 - Artikel entfernen
3 - Liste anzeigen
4 - Beenden
```

```
Auswahl: 1
Artikel: Milch
'Milch' hinzugefügt.
```

```
Auswahl: 3
1. Milch
2. Brot
```

```
Auswahl: 2
Artikel entfernen: Zucker
'Zucker' ist nicht in der Liste.
```

```
Auswahl: 4
Auf Wiedersehen!
```

Aufgabe 4: Playlist verwalten

Beschreibung: Eine Musikplaylist ist nichts anderes als eine geordnete Liste und genau das lässt sich mit den neuen Listenkenntnissen prima umsetzen.

Aufgabenstellung: Schreibe ein Playlist-Programm mit einem Menü. Der Benutzer kann folgende Optionen wählen:

- 1 – Song hinzufügen (ans Ende)
- 2 – Song entfernen (nach Name)
- 3 – Song nach oben verschieben (einen Platz)
- 4 – Playlist anzeigen
- 5 – Beenden

Beim Verschieben nach oben soll der Song mit dem Song davor die Position tauschen. Ist der Song bereits an erster Stelle, soll eine passende Meldung ausgegeben werden.

Beispielausgabe:

```
=== Meine Playlist ===
```

```
1 - Song hinzufügen
2 - Song entfernen
3 - Song nach oben verschieben
4 - Playlist anzeigen
5 - Beenden
```

```
Auswahl: 1
```

```
Song: Bohemian Rhapsody
```

```
'Bohemian Rhapsody' hinzugefügt.
```

```
Auswahl: 4
```

```
1. Bohemian Rhapsody
2. Stairway to Heaven
3. Hotel California
```

```
Auswahl: 3
```

```
Song nach oben verschieben: Hotel California
```

```
'Hotel California' ist jetzt auf Platz 2.
```

```
Auswahl: 4
```

```
1. Bohemian Rhapsody
2. Hotel California
3. Stairway to Heaven
```

```
Auswahl: 5
```

```
Playlist gespeichert. Bis zum nächsten Mal!
```

Aufgabe 5: Duplikate entfernen

Aufgabenstellung: Schreibe ein Programm, das den Benutzer so lange nach Zahlen fragt, bis er eine leere Eingabe macht. Danach soll das Programm:

- Die eingegebene Liste mit allen Werten ausgeben
- Eine bereinigte Liste ohne doppelte Werte ausgeben
- Ausgeben, welche Werte doppelt eingegeben wurden

Verwende ausschließlich den `in`-Operator und eine Schleife, keine eingebauten Funktionen zum Entfernen von Duplikaten.

Beispielausgabe:

```
Zahl eingeben (Enter zum Beenden): 5
Zahl eingeben (Enter zum Beenden): 3
Zahl eingeben (Enter zum Beenden): 8
Zahl eingeben (Enter zum Beenden): 3
Zahl eingeben (Enter zum Beenden): 5
Zahl eingeben (Enter zum Beenden): 9
Zahl eingeben (Enter zum Beenden):
```

```
Eingabe:  [5, 3, 8, 3, 5, 9]
Bereinigt: [5, 3, 8, 9]
Doppelt:  [5, 3]
```

[Bonus] Taschenrechner erweitern

Beschreibung: Der Taschenrechner wächst mit jeder Einheit weiter. In Level 06 war er einmalig, in Level 07 lief er in einer Schleife. Jetzt kennst du Listen. Überlege dir, wie du den Taschenrechner damit sinnvoll erweitern kannst. Es gibt keine feste Vorgabe, was genau umzusetzen ist. Lass dich von den Anstößen unten inspirieren und wähle aus, was dir interessant erscheint.

Aufgabenstellung: Nimm den Taschenrechner aus Level 07 als Ausgangspunkt und erweitere ihn mit mindestens einer der folgenden Ideen:

- **Verlauf:** Speichere jede Berechnung als Eintrag in einer Liste und zeige den Verlauf auf Wunsch oder am Ende an.
- **Letztes Ergebnis übernehmen:** Biete dem Benutzer an, das Ergebnis der letzten Berechnung direkt als erste Zahl der nächsten zu verwenden.
- **Ergebnis rückgängig machen:** Ermögliche es, den letzten Eintrag aus dem Verlauf zu entfernen.
- **Statistik:** Zeige am Ende aus, wie viele Berechnungen durchgeführt wurden und was das größte und kleinste Ergebnis war.

Du darfst auch eigene Ideen umsetzen oder mehrere Punkte kombinieren.

[Bonus] Hangman / Galgenmännchen

Beschreibung: Hangman ist ein klassisches Ratespiel und lässt sich mit Listen elegant umsetzen. Das gesuchte Wort wird als Liste von Buchstaben gespeichert, bereits geratene Buchstaben in einer zweiten Liste. Es gibt keine feste Vorgabe für das Aussehen des Spiels. Entwirf deine eigene Version und ergänze, was dir Spaß macht.

Aufgabenstellung: Schreibe dein eigenes Hangman-Spiel. Typische Regeln und Bestandteile eines Hangman-Spiels sind:

- Ein gesuchtes Wort wird Buchstabe für Buchstabe erraten
- Noch nicht geratene Buchstaben werden als `_` angezeigt
- Der Spieler hat eine begrenzte Anzahl an Fehlversuchen
- Bereits geratene Buchstaben werden erkannt und gemeldet
- Das Spiel endet mit Sieg (Wort vollständig erraten) oder Niederlage (zu viele Fehlversuche)

Erweiterungsideen:

- Wähle das gesuchte Wort zufällig aus einer Liste von Wörtern
- Zeige nach jedem Fehlversuch eine ASCII-Grafik des Galgens, dazu findest du im Internet viele fertige Vorlagen, z.B. wenn du nach „hangman ascii art python“ suchst
- Zeige am Ende alle falsch geratenen Buchstaben an
- Füge verschiedene Schwierigkeitsstufen mit unterschiedlich langen Wörtern hinzu

[Bonus] Bubblesort

Beschreibung: Sortieralgorithmen gehören zu den grundlegenden Konzepten der Informatik. Die eingebaute `sort()` -Methode erledigt das Sortieren automatisch, aber wie funktioniert Sortieren eigentlich? Bubblesort ist einer der einfachsten Algorithmen und lässt sich gut mit Listen und Schleifen umsetzen.

Aufgabenstellung: Schreibe ein Programm, das eine Liste von 8 ganzen Zahlen (direkt im Code festgelegt) mit dem Bubblesort-Algorithmus sortiert, ohne die eingebaute `sort()` -Methode zu verwenden.

Bubblesort: Gehe die Liste wiederholt durch. Vergleiche immer zwei benachbarte Elemente. Sind sie in der falschen Reihenfolge, tausche sie. Wiederhole das, bis kein Tausch mehr nötig ist.

Gib nach jedem vollständigen Durchlauf den aktuellen Zustand der Liste aus.

Hinweis: Zum Tauschen zweier Listenelemente kannst du eine Hilfsvariable verwenden:

```
temp      = liste[i]
liste[i]   = liste[i + 1]
liste[i + 1] = temp
```

Beispielausgabe:

Ausgangsliste: [5, 3, 8, 1, 9, 2, 7, 4]

Durchlauf 1: [3, 5, 1, 8, 2, 7, 4, 9]

Durchlauf 2: [3, 1, 5, 2, 7, 4, 8, 9]

Durchlauf 3: [1, 3, 2, 5, 4, 7, 8, 9]

Durchlauf 4: [1, 2, 3, 4, 5, 7, 8, 9]

Durchlauf 5: [1, 2, 3, 4, 5, 7, 8, 9]

Sortierte Liste: [1, 2, 3, 4, 5, 7, 8, 9]